Diario SQL

Emma Choque Sandoval

Introducción

El año pasado cursé Bases de Datos en el grado superior, así que SQL no me es nuevo. Por eso me ubiqué en el Nivel 2 (Analista SQL): lo básico lo manejo bien y lo que quería era afinar las funciones de agregación y los agrupamientos, que es donde más me trabo.
Trabajé sobre la base soc_db de la herramienta, que simula un centro de operaciones de seguridad. Tiene tres tablas:

- usuarios (id_usuario, nombre, departamento, email): 20 usuarios.
- dispositivos (id_dispositivo, id_usuario, tipo_dispositivo, sistema_operativo): 20 dispositivos.
- eventos_login (id_evento, id_usuario, id_dispositivo, ip_origen, fecha_hora, resultado): 40 eventos de login.

Parte 1. Mi ruta de aprendizaje individual

Misión 1. Consigue tus primeras pistas

Al terminar, comprueba que sabes: consultar con SELECT, indicar la tabla con FROM, filtrar con WHERE, combinar condiciones con AND y OR y ordenar con ORDER BY.
Todo eso lo tengo claro. Lo practiqué con las tablas de soc_db: SELECT para elegir columnas, FROM para decir de qué tabla, WHERE para filtrar filas, AND y OR para juntar condiciones y ORDER BY para ordenar (ASC de menor a mayor, DESC de mayor a menor).
Evidencia: escribe dos consultas que hayas resuelto y explica qué hace cada una.
Consulta 1 (ejercicio "Nombre y departamento")

```
SELECT nombre, departamento
FROM usuarios;
```

Qué hace: me muestra solo las columnas nombre y departamento de los 20 usuarios. No uso * porque solo necesito esas dos. Las primeras filas son Ana Torres (Finanzas), Luis García (TI), María López (RRHH), Carlos Ruiz (Ventas) y Sofía Núñez (Marketing).
Consulta 2 (ejercicio "Usuarios de Finanzas")

```
SELECT nombre, email
FROM usuarios
WHERE departamento = 'Finanzas'
ORDER BY nombre;
```

Qué hace: se queda solo con los usuarios del departamento de Finanzas

y los ordena por nombre. Devuelve dos filas: Ana Torres (ana.torres@corp.local) y Diego Pérez (diego.perez@corp.local).

Misión 2. Sube de nivel

Practica: seleccionar solo las columnas necesarias, filtrar por varias condiciones, ordenar, COUNT(), SUM() y GROUP BY.
Practiqué todo menos SUM() con algo que tuviera sentido, porque en soc_db no hay ninguna columna numérica para sumar (los id no se suman). Con COUNT(), GROUP BY y los filtros sí trabajé bastante.
Reto de comprensión: antes de ejecutar una consulta, predice qué resultado devolverá y compáralo con el real.
Consulta (ejercicio "Fallos por IP"):

```
SELECT ip_origen, COUNT(*) AS total_fallos
FROM eventos_login
WHERE resultado = 'fallo'
GROUP BY ip_origen
ORDER BY total_fallos DESC;
```

Mi predicción: que la IP externa 203.0.113.77 iba a tener muchos más fallos que las demás, y que las IP internas (10.0.0.x) iban a tener pocos. Resultado real: 203.0.113.77 tiene 7 fallos y las IP 10.0.0.25, 10.0.0.18 y 10.0.0.17 tienen 1 fallo cada una. Acerté en lo principal, pero no esperaba que hubiera tres IP internas con fallos sueltos.
Evidencia: documenta una consulta que te haya costado resolver y explica cómo lo conseguiste.
Me costó la de "Usuario más atacado":

```
SELECT id_usuario, COUNT(*) AS total_fallos
FROM eventos_login
WHERE resultado = 'fallo'
GROUP BY id_usuario
ORDER BY total_fallos DESC
LIMIT 1;
```

Resultado: id_usuario 6 con 7 fallos. Cómo lo conseguí: primero filtro los fallos con WHERE (eso va antes de agrupar), después agrupo por usuario con GROUP BY, cuento con COUNT(*), ordeno de mayor a menor con ORDER BY ... DESC y me quedo solo con la primera fila con LIMIT 1. Lo que me costó fue el orden de las cláusulas: al principio quise poner el filtro de fallos después del GROUP BY y me daba error. Releí la teoría y vi que WHERE siempre va antes de GROUP BY. Después, con la tabla usuarios, vi que el id 6 es Diego Pérez.

Misión 3. Entra en SQL Agent

Elige un reto que te suponga esfuerzo, intenta resolverlo sin mirar la solución, usa pistas si te bloqueas, explica por qué funciona y, si terminas pronto, elige otro más difícil.
Elegí "Ranking de víctimas con nombre", que es de los difíciles porque mezcla JOIN con agrupación. Lo intenté sin mirar la solución, me trabé

un rato con el JOIN y volví a leer la teoría del módulo 6 antes de resolverlo. El detalle de la consulta y por qué funciona está en el Reto de SQL Agent del entregable, más abajo.
Regla de oro: no copié ninguna solución sin entenderla. Si no puedo explicarla con mis palabras, no la doy por aprendida.

Misión 4. Mi registro de aprendizaje

- **Una consulta SQL sirve para …** pedirle datos a una base de datos de forma ordenada y filtrada, sin tener que revisar fila por fila a mano. En soc_db, por ejemplo, para ver quién está recibiendo intentos de login fallidos.
- **La instrucción que mejor entiendo es …** WHERE junto con ORDER BY. Las uso todo el tiempo y ya las tengo automáticas.
- **La instrucción que todavía me cuesta es …** JOIN, sobre todo escribir bien la condición del ON y no mezclar las columnas de las dos tablas.
- **El error más interesante que he encontrado es …** poner el filtro de fallos en el lugar equivocado al usar GROUP BY. Gracias a eso entendí el orden en que SQL procesa la consulta: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY y LIMIT.
- **Una cosa que ahora puedo hacer y antes no sabía hacer es …** detectar un posible ataque con SQL: ver que una IP externa tuvo 7 fallos seguidos contra un mismo usuario y saber de quién se trata.

Lista de comprobación individual

- ☑ He trabajado los contenidos introductorios.
- ☑ He ejecutado consultas y observado sus resultados.
- ☑ He intentado resolver ejercicios sin copiar directamente.
- ☑ He probado al menos un reto avanzado o una actividad de refuerzo.
- ☑ He documentado mis aprendizajes y mis dudas.

Parte 2. Elige tu siguiente nivel

Elegí el Nivel 2: Analista SQL (ya controlo lo básico). No es una competición, pero con la base del año pasado me pareció el que más me aportaba.
1. Cuenta cuántos registros tiene una tabla.

```
SELECT COUNT(*) AS total_eventos
FROM eventos_login;
```

Resultado: 40. COUNT(*) cuenta todas las filas de la tabla, o sea los 40 eventos de login. Con usuarios pasa lo mismo y da 20.
2. Calcula la suma de una columna numérica.
Acá tengo que ser sincera: en soc_db no hay una columna numérica que tenga sentido sumar. Las únicas numéricas son los id, y sumar ids no

dice nada. Por eso, para practicar la idea de "calcular un total", usé COUNT con filtro, que es lo que más se parece. Si hubiera una columna como "bytes" o "intentos", usaría SUM(columna) del mismo modo.

```
SELECT COUNT(*) AS total_fallos
FROM eventos_login
WHERE resultado = 'fallo';
```

Resultado: 10 fallos en total.

3. Agrupa los registros por una categoría.

```
SELECT resultado
FROM eventos_login
GROUP BY resultado;
```

Resultado: dos filas, exito y fallo. GROUP BY junta las filas que tienen el mismo valor, así que me queda una fila por categoría.

4. Cuenta cuántos registros hay en cada categoría.

```
SELECT resultado, COUNT(*) AS total
FROM eventos_login
GROUP BY resultado;
```

Resultado: exito 30 y fallo 10. Los 30 más los 10 suman los 40 eventos, así que me sirve para comprobar que no me falta nada.

5. Ordena los grupos para identificar el mayor o el menor.

```
SELECT ip_origen, COUNT(*) AS total_fallos
FROM eventos_login
WHERE resultado = 'fallo'
GROUP BY ip_origen
ORDER BY total_fallos DESC;
```

Resultado: la primera fila es 203.0.113.77 con 7 fallos, que es el grupo mayor. Si cambio DESC por ASC, la primera sería una de las IP con 1 fallo, que es el menor.

Si ya conoces HAVING, úsalo para filtrar los grupos después de calcular sus resultados.

```
SELECT ip_origen, COUNT(*) AS total_fallos
FROM eventos_login
WHERE resultado = 'fallo'
GROUP BY ip_origen
HAVING COUNT(*) > 1;
```

Resultado: solo 203.0.113.77 con 7 fallos. Las IP con un único fallo quedan fuera. Usé WHERE para filtrar filas antes de agrupar y HAVING para filtrar los grupos después de contar.

Condición para avanzar: resolver una pregunta utilizando una función de agregación y explicar su resultado.

Pregunta: ¿cuántos intentos fallidos hubo en total y cuántos vinieron de una IP externa? Con la consulta de arriba veo 10 fallos en total, y 7 vienen de 203.0.113.77, una IP que no es de la red interna 10.0.0.x. O sea, 7 de cada 10 fallos son del mismo origen externo, y eso es lo que haría saltar una alerta en un SOC.

Parte 4. Autoevaluación final

- ☑ Sé consultar datos de una tabla.
- ☑ Sé filtrar y ordenar resultados.
- ☑ Comprendo para qué sirven las funciones de agregación.
- ☑ He intentado al menos un reto de SQL Agent.
- ☑ He guardado ejemplos de consultas y mis aprendizajes.
- ☑ Puedo explicar al menos una consulta que he escrito.
- ☑ He identificado qué necesito seguir practicando.

Entregable

1. Registro de consultas SQL

Cada alumna guardará 3 consultas realizadas en SQL Case Files, con el código SQL, el resultado y una explicación breve de cómo funciona.

Consulta 1: básica que comprendí bien ("Nombre y departamento")

```
SELECT nombre, departamento
FROM usuarios;
```

Resultado: una tabla de dos columnas (nombre y departamento) con una fila por cada uno de los 20 usuarios, por ejemplo Ana Torres - Finanzas y Luis García - TI.

Cómo funciona: SELECT indica qué columnas quiero ver y FROM de qué tabla las saco. Es la base de todo lo demás.

Consulta 2: con filtro y ordenación ("Usuarios de Finanzas")

```
SELECT nombre, email
FROM usuarios
WHERE departamento = 'Finanzas'
ORDER BY nombre;
```

Resultado: dos filas, Ana Torres (ana.torres@corp.local) y Diego Pérez (diego.perez@corp.local).

Cómo funciona: WHERE descarta las filas que no son de Finanzas y después ORDER BY ordena las que quedaron por nombre, de la A a la Z.

Consulta 3: la que fue un reto ("Usuario más atacado")

```
SELECT id_usuario, COUNT(*) AS total_fallos
FROM eventos_login
WHERE resultado = 'fallo'
GROUP BY id_usuario
ORDER BY total_fallos DESC
LIMIT 1;
```

Resultado: id_usuario 6 con 7 fallos, que corresponde a Diego Pérez, de Finanzas.

Cómo funciona: WHERE se queda solo con los eventos fallidos, GROUP BY los junta por usuario, COUNT(*) cuenta cuántos fallos tiene cada uno, ORDER BY los ordena del que más tiene al que menos y LIMIT 1 deja solo el primero. Lo que me costó fue el orden de las cláusulas.

2. Reto de SQL Agent

Documentará al menos un reto que le haya supuesto esfuerzo.

¿Qué pedía el reto?

El reto era "Ranking de víctimas con nombre": hacer un ranking de los usuarios con más intentos fallidos de login, pero mostrando su nombre y no solo el id.

¿Qué consulta ha escrito?

```
SELECT u.nombre, COUNT(*) AS total_fallos
FROM eventos_login e
JOIN usuarios u ON e.id_usuario = u.id_usuario
WHERE e.resultado = 'fallo'
GROUP BY u.nombre
ORDER BY total_fallos DESC;
```

¿Ha conseguido resolverlo?

Sí, aunque no a la primera. Al principio me olvidé de poner alias a las tablas (e y u) y no sabía a cuál pertenecía cada columna. Cuando entendí que el ON dice por qué columna se relacionan las dos tablas (id_usuario en las dos), salió. El resultado fue: Diego Pérez 7, y María López, Marta Ríos y Carlos Ruiz con 1 cada uno.

¿Qué ha aprendido al resolverlo o al revisar el resultado?

- Que JOIN une las filas de dos tablas usando una columna en común, y que el ON es lo que decide cómo se emparejan.
- Que puedo combinar JOIN con WHERE, GROUP BY y ORDER BY en una sola consulta, y que el orden es siempre el mismo: FROM y JOIN, WHERE, GROUP BY, ORDER BY.
- Que los alias (e y u) hacen la consulta más corta y evitan confusiones cuando dos tablas tienen columnas con el mismo nombre.
- Que el resultado cuenta una historia: Diego Pérez recibió 7 fallos seguidos desde una IP externa en pocos minutos, de madrugada, y después hubo un login con éxito. Eso en un SOC sería sospechoso de una cuenta comprometida.

3. Reflexión final

¿Qué concepto de SQL entiendo mejor ahora?

GROUP BY con COUNT. Antes lo usaba de memoria y ahora entiendo qué pasa: se arman grupos con las filas que tienen el mismo valor y la función calcula un resultado por cada grupo.

¿Qué ejercicio me ha resultado más difícil?

"Ranking de víctimas con nombre", por el JOIN. Tuve que pensar bien qué columna relacionaba las dos tablas y en qué orden iban las cláusulas.

¿Qué puedo hacer ahora que al principio no sabía hacer?

Resumir información con GROUP BY, combinar dos tablas con JOIN y ordenar para ver el mayor, todo en una sola consulta. Y también leer los resultados para sacar una conclusión, como detectar una IP

sospechosa.

¿Qué necesito seguir practicando?

JOIN con más de dos tablas, por ejemplo unir eventos_login con usuarios y dispositivos a la vez, y también HAVING. Además quiero practicar CREATE TABLE, INSERT y DELETE, que son los últimos ejercicios y los que menos toqué.